



Today's agenda

↳ LPSL construction

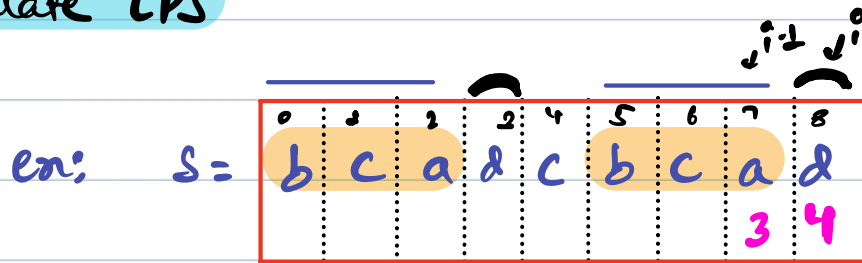
↳ multiply 2 strings



AlgoPrep

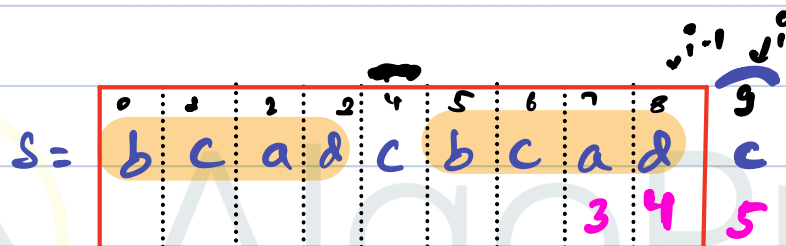


↳ calculate LPS

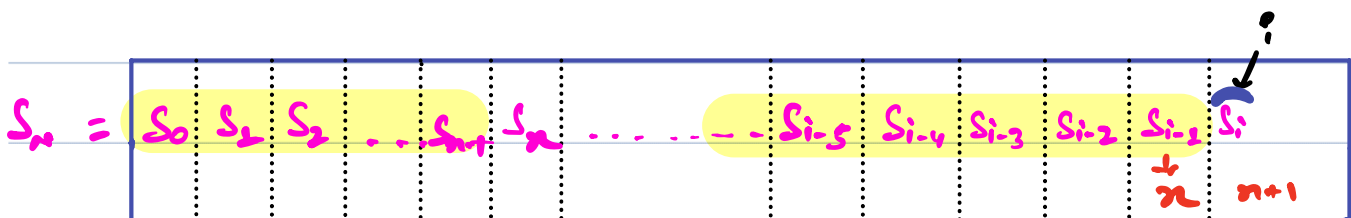


OBS 1: The LPS value $(i+1)$ th index can never be more than 1 compared to i th index.

↳ The minimum LPS value can be 0.



// General



if $(S_i == S_n)$ {
 $LPS[i] = LPS[i-1] + 1;$
 }

Arrows indicate the update of $LPS[i]$ from $LPS[i-1]$.



6

S =

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
c	a	c	y	c	a	c	a	b	c	a	c	y	c	a	c	y
0	0	1	0	1	2	3	2	0	1	2	3	4	5	6	7	

i = 16

len = 7

ans | S[i] == S[len]

8 | S[16] == S[7] → not same → LPS[i] = 8

4 | S[16] == S[3]

↳ matches → LPS[i] = 4

8 length x x

7 length

6 length

1 length

0 length



Prefix

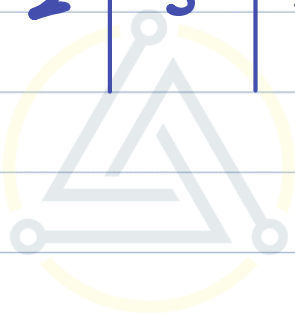
S:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
a	b	c	a	b	d	a	b	c	a	b	e	a	b	c	a	b	d	a	b	c	a	b	c
0	0	0	1	2	0	1	2	3	4	5	0	1	2	3	4	5	6	7	8	9	10	11	3

i = 23

len = 11

len	ans	$S[i] == S[len]$
11	12	$S[23] == S[11] \rightarrow \text{Not matching}$
5	6	$S[23] == S[5] \rightarrow \text{Not matching}$
2	3	$S[23] == S[2] \rightarrow \text{matching} \rightarrow LPS[i] = 3$



AlgoPrep



1/1 Pseudo code

```
int[] LPS (String s(N)) {  
    int LPS[N];
```

```
    LPS[0] = 0;
```

```
    for (int i = 1; i < N; i++) {
```

```
        int len = LPS[i-1];
```

```
        if (s[i] == s[len]) {
```

```
            LPS[i] = len + 1;
```

```
        }
```

```
        else {
```

```
            while (s[i] != s[len]) {
```

```
                if (len == 0) { len = -1; break; }
```

```
                len = LPS[len-1];
```

```
            }
```

```
            LPS[i] = len + 1;
```

```
        }
```

```
    }
```

```
    return LPS;
```

```
}
```

T.C: $O(N)$

S.C: $O(1) / O(N)$



```

for (int i=0; i<N; i++) {
    for (int j=0; j<i; j++) {
        // ...
    }
}

```

$\rightarrow O(N^2)$

i	j	Count
0	-	0
1	[0,0]	1
2	[0,1]	2
3	[0,2]	3
...	...	...
N-1	[0, N-2]	N-1
		$\frac{N*(N-1)}{2} = O(N^2)$

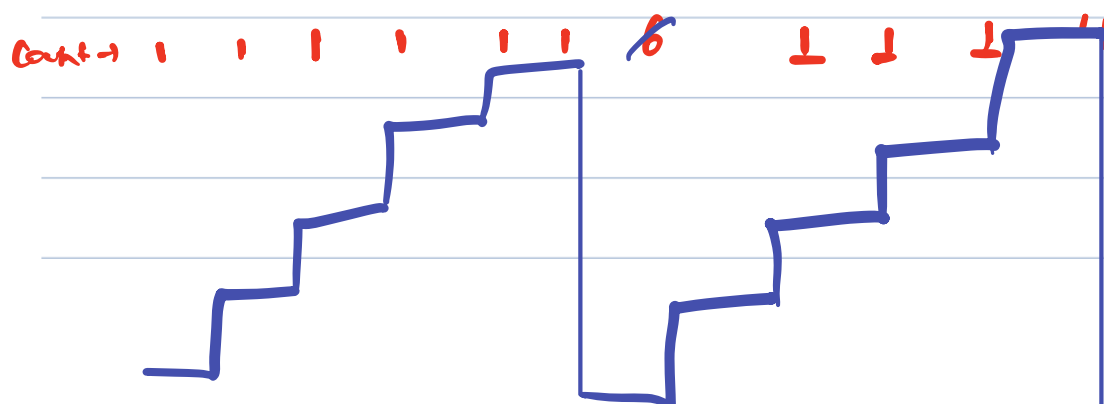
```

for (int i=1; i<N; i++) {
    int len = LPS[i-1];
    if (s[i] == s[len]) {
        LPS[i] = len+1;
    }
    else {
        while (s[i] != s[len]) {
            if (len == 0) { len = -1; break; }
            len = LPS[len-1];
        }
        LPS[i] = len+1;
    }
}

```

i	j	Count
0		0
1		1
2		1/2
3		1/2/3
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮
		⋮

LPS $\rightarrow$ 0 1 2 3 4 5 ~~6~~ 1 2 3 ~~4~~ $\Rightarrow O(N)$





i

	0	1	2	3	4	5	6	7	8	9	10	11
S:	a	b	c	a	b	d	a	b	c	a	b	e
	0	0	0	1	2	0	1	2	3	4	5	0

```
for(int i=1; i<=n; i++) {
    int len = LPS[i-1];
    if (s[i] == s[len]) {
        LPS[i] = len+1;
    }
    else {
        while (s[i] != s[len]) {
            if (len == 0) { len = -1; break; }
            len = LPS[len-1];
        }
        LPS[i] = len+1;
    }
}
```

i	len
1	0-1
2	0-1
3	0
4	1
5	2 → 0-1
6	0
7	1
8	2
9	3
10	4
11	5 → 2 → 0-1